

Stack in Java - Implementation and Usage

What is a Stack?

- A Stack is a linear data structure that follows the Last-In-First-Out (LIFO) principle.
- The element that is added last is accessed first.
- Common operations include push, pop, peek, search, and empty.

Java Stack Class

- Java provides a built-in Stack class under the java.util package.
- The Stack class extends Vector and supports operations like push, pop, peek, search, and empty.

Syntax:

```
import java.util.Stack;
```

```
public class StackExample {  
  
    public static void main(String[] args) {  
  
        Stack<Integer> stack = new Stack<>();  
  
  
        // Push elements to the stack  
  
        stack.push(10);  
  
        stack.push(20);  
  
        stack.push(30);  
  
  
        // Peek at the top element  
  
        System.out.println("Top element: " + stack.peek()); // 30  
  
  
        // Pop an element  
  
        System.out.println("Popped element: " + stack.pop()); // 30
```

```

// Check if stack is empty

System.out.println("Is stack empty? " + stack.empty()); // false


// Search for an element

System.out.println("Position of 10: " + stack.search(10)); // 2

}

}

```

Common Stack Methods:

- **push(E item):** Adds an item to the top of the stack.
- **pop():** Removes and returns the top item of the stack.
- **peek():** Returns the top item without removing it.
- **empty():** Checks if the stack is empty.
- **search(Object o):** Returns the position of the item from the top (1-based index).

Additional Stack Methods:

- **clone():** Creates a shallow copy of the stack.
- **remove(int index):** Removes the element at the specified position (inherited from Vector).
- **contains(Object o):** Checks if the stack contains a specific element (inherited from Vector).

```
package Stack_Examples;

import java.util.Scanner;
import java.util.Stack;

public class Stack_Built_in_Example
{
    public static void main(String args[])
    {
        Scanner in=new Scanner(System.in);
        Stack<Integer> obj=new Stack<>();//object
        int ch;
        Integer e;
        do
        {
            System.out.println("Stack Menu");
            System.out.println("-----");
            System.out.println("1.Push");
            System.out.println("2.Pop");
            System.out.println("3.Peek");
            System.out.println("4.Print all elements");
            System.out.println("0.Exit");
```

```

System.out.println("-----");

System.out.println("Choice:");

ch=in.nextInt();

switch(ch)
{
    case 1:

        System.out.println("Enter a number:");

        e=in.nextInt();

        obj.push(e);

        break;

    case 2:

        if(obj.empty()!=true)
        {
            System.out.println("Element popped is
"+obj.pop());
        }

        else {

            System.out.println("Stack Empty");

        }

        break;

    case 3:

        if(obj.empty()!=true) //not empty
        {

            System.out.println("Element @peek is
"+obj.peek());

        }

```

```

else {
    System.out.println("Stack Empty");
}

break;

case 4:

    if(obj.empty()!=true)//not empty
    {
        System.out.println("Elements in stack
are\n");

        /*option 1 no LIFO
        for (Integer element:obj)
        {
            System.out.println(element);
        }

        //print all
        // System.out.println(obj);*/

        for(int
index=obj.size()-1;index>-1;index--)//LIFO Print

            System.out.println(obj.get(index));
    }

    else {
        System.out.println("Stack Empty");
    }

    break;

case 0:

    System.out.println("Thanks for using
code");

```

```
        break;

    default:

        System.out.println("Wrong option
selected:");

        break;

    }

}while(ch!=0);

}

}
```

2. Next Greater Element

Problem: Given an array of distinct integers, for each element in the array, find the next greater element. The next greater element for an element x is the first element to its right that is greater than x . If no such element exists, the next greater element should be -1 .

Algo:

Traverse the Array from Right to Left:

- For each element, while the stack is not empty and the top of the stack is less than or equal to the current element, pop the stack (because the current element is a more suitable candidate for the next greater element for other elements to its left).
- If the stack is not empty, the top of the stack is the next greater element for the current element.
- If the stack is empty, the next greater element is -1 .
- Push the current element onto the stack.

Input Sample:

[4, 5, 2, 10, 8]

Output Sample:

[5, 10, 10, -1 , -1]

Explanation of Output:

- Next greater element of 4 is 5.
- Next greater element of 5 is 10.
- Next greater element of 2 is 10.
- Next greater element of 10 is -1 (no greater element to its right).
- Next greater element of 8 is -1 (no greater element to its right).

Input Sample:

[1, 2, 3, 4]

Output Sample:

[2, 3, 4, -1]

Stock Span Problem

Problem: The stock span problem is a financial market technical analysis tool that indicates the number of consecutive days preceding the current day for which the stock price has been less than or equal to the stock price on the current day. Given an array of stock prices for consecutive days, calculate the span for each day.

Explanation: For each day, we need to find the number of consecutive days before it (including the current day) where the stock price was less than or equal to the current day's price. A stack can be used to efficiently solve this. We iterate through the stock prices. For each day's price, we compare it with the prices of the previous days stored in the stack.

1. Create a result array to store the spans.
2. Initialize an empty stack. The stack will store the indices of the days.
3. Iterate through the stock prices from left to right (for each day i):
 - While the stack is not empty and the stock price at the index on top of the stack is less than or equal to the current day's stock price ($\text{prices}[\text{stack.peek()}] \leq \text{prices}[i]$): pop the index from the stack.
 - If the stack becomes empty, it means the current day's price is greater than all the previous day's prices, so the span is $i + 1$.
 - If the stack is not empty, the span is the difference between the current day's index and the index at the top of the stack ($i - \text{stack.peek()} + 1$).
 - Push the current day's index i onto the stack.
4. Return the result array containing the spans.

Input Sample:

[100, 80, 60, 70, 60, 75, 85]

Output Sample:

[1, 1, 1, 2, 1, 4, 6]

Explanation of Output:

- Day 1 (price 100): Span is 1 (no previous day).
- Day 2 (price 80): Span is 1 (80 is not greater than 100).
- Day 3 (price 60): Span is 1 (60 is not greater than 80).
- Day 4 (price 70): Span is 2 (70 > 60, 70 is not greater than 80).
- Day 5 (price 60): Span is 1 (60 is not greater than 70).
- Day 6 (price 75): Span is 4 (75 > 60, 75 > 70, 75 > 60, 75 is not greater than 80).
- Day 7 (price 85): Span is 6 (85 > 75, 85 > 60, 85 > 70, 85 > 60, 85 > 80).

Queue in Java - Implementation and Methods

What is a Queue?

- A Queue is a linear data structure that follows the First-In-First-Out (FIFO) principle.
- The element that is added first is removed first.
- Common operations include enqueue (add), dequeue (remove), peek, and size.

Java Queue Interface

- Java provides the Queue interface under the java.util package.
- Common implementations include LinkedList, PriorityQueue, and ArrayDeque.

Syntax:

```
import java.util.Queue;

import java.util.LinkedList;

public class QueueExample {

    public static void main(String[] args) {

        Queue<Integer> queue = new LinkedList<>();

        // Add elements to the queue

        queue.add(10);

        queue.add(20);

        queue.add(30);

        // Peek at the front element

        System.out.println("Front element: " + queue.peek()); //
```

```

        // Remove an element

        System.out.println("Removed element: " + queue.poll());
// 10

        // Check size of the queue

        System.out.println("Size of queue: " + queue.size()); //
2
    }

}

```

Common Queue Methods:

- `add(E e)`: Inserts the specified element at the end of the queue.
- `offer(E e)`: Inserts the specified element; returns false if the queue is full (non-blocking).
- `remove()`: Removes and returns the head of the queue; throws exception if empty.
- `poll()`: Removes and returns the head of the queue; returns null if empty.
- `element()`: Retrieves the head without removing it; throws exception if empty.
- `peek()`: Retrieves the head without removing it; returns null if empty.
- `size()`: Returns the number of elements in the queue.
- `contains(Object o)`: Checks if the queue contains a specific element.
- `iterator()`: Returns an iterator over the queue elements.

Question:

Generate Binary Numbers from 1 to N

Problem: Given a positive integer N, generate binary numbers from 1 to N using a queue.

Explanation:

We can use a queue to generate binary numbers in order.

1. Enqueue "1" into the queue.
2. While N is greater than 0:
 - Dequeue a binary number from the queue.
 - Print the dequeued number.
 - Enqueue the dequeued number appended with "0" and "1" into the queue.
 - Decrement N.

Input Sample:

N = 5

Output Sample:

1 10 11 100 101

Reverse First K Elements of Queue

Problem: Given a queue and an integer K, reverse the order of the first K elements of the queue, leaving the order of the remaining elements unchanged.

Explanation: We can use a stack to reverse the first K elements.

1. Dequeue the first K elements from the queue and push them onto a stack.
2. Pop the elements from the stack and enqueue them back into the queue. This reverses the order of the first K elements.
3. Dequeue the remaining elements (n-K) from the queue and enqueue them back into the queue.

Input Sample:

Queue: [1, 2, 3, 4, 5], K = 3

Output Sample: Queue: [3, 2, 1, 4, 5]

Circular Tour Problem

Problem: Suppose there is a circular track. There are N petrol pumps on this track. You are given two arrays: $\text{petrol}[i]$ is the amount of petrol that the i -th petrol pump has, and $\text{distance}[i]$ is the distance from the i -th petrol pump to the next petrol pump. You start with an empty tank. Find the index of the starting petrol pump from where you can complete the circular tour.

Explanation: We can use a queue to simulate the circular tour.

1. Enqueue all petrol pumps into the queue.
2. Simulate the tour starting from the front of the queue.
3. Keep track of the current petrol in the tank.
4. If the current petrol becomes negative at any point, it means the current starting point is not valid. Dequeue the current starting point and try with the next petrol pump in the queue.
5. If we can complete the tour, return the starting index.
6. If we have tried all the petrol pumps and still cannot complete the tour, return -1.

Petrol: [6, 3, 7] **Distance:** [4, 6, 3]

Explanation:

- Start at Pump 0:
 - Petrol: $6 - 4 = 2$ (Remaining petrol)
- Move to Pump 1:
 - Petrol: $2 + 3 - 6 = -1$ (Not enough petrol to proceed)
 - Restart from Pump 1.
- Start at Pump 1:
 - Petrol: $3 - 6 = -3$ (Not enough petrol to proceed)
 - Restart from Pump 2.

- Start at Pump 2:
 - Petrol: $7 - 3 = 4$
 - Move to Pump 0: $4 + 6 - 4 = 6$
 - Move to Pump 1: $6 + 3 - 6 = 3$ (Completed the circuit successfully)

```
package DSA_Problems;
```

```
public class Circular_Tour_Problem
```

```
{
```

```
    static int findStartingPoint(int petrol[],int
distance[],int start_from)
```

```
    {
```

```
        int n=petrol.length;
```

```
        int current_petrol=0;
```

```
        int round=0;
```

```
        for(int i=start_from;round<n;i=(i+1)%n)
```

```
        {
```

```
            current_petrol+=petrol[i]-distance[i];
```

```
            round++;
```

```
            if(current_petrol<0)
```

```
                return
```

```
findStartingPoint(petrol,distance,start_from+1);
```

```
        }
```

```
        return start_from;
```

```
}
```



```
public static void main(String[] args) {  
    // Sample Input  
    int[] petrol = {6, 3, 7};  
    int[] distance = {4, 6, 3};  
    int startIndex = findStartingPoint(petrol,  
distance,0);  
    System.out.println("Starting Pump Index: " +  
startIndex);  
}  
  
}
```

LinkedList in Java - Implementation and Methods

What is a LinkedList?

- A LinkedList is a linear data structure consisting of nodes where each node contains data and a reference to the next node.
- Java provides the LinkedList class as part of the java.util package.
- It implements both List and Deque interfaces, allowing it to function as a list, queue, or deque.

Syntax:

```
import java.util.LinkedList;
```

```
public class LinkedListExample {  
  
    public static void main(String[] args) {  
  
        LinkedList<Integer> list = new LinkedList<>();  
  
  
        // Adding elements  
  
        list.add(10);  
  
        list.add(20);  
  
        list.add(30);  
  
  
        // Accessing elements  
  
        System.out.println("First Element: " + list.getFirst());  
// 10  
  
        System.out.println("Last Element: " + list.getLast());  
// 30
```

```

        // Removing elements

        list.removeFirst();

        System.out.println("After removing first: " + list);

    }

}

```

Common LinkedList Methods:

- `add(E e)`: Adds an element to the end of the list.
- `addFirst(E e)`: Adds an element at the beginning of the list.
- `addLast(E e)`: Adds an element at the end of the list.
- `get(int index)`: Returns the element at the specified index.
- `getFirst()`: Returns the first element.
- `getLast()`: Returns the last element.
- `remove(int index)`: Removes the element at the specified index.
- `removeFirst()`: Removes the first element.
- `removeLast()`: Removes the last element.
- `contains(Object o)`: Checks if the list contains a specific element.
- `size()`: Returns the number of elements in the list.
- `clear()`: Removes all elements from the list.
- `indexOf(Object o)`: Returns the index of the first occurrence of the specified element.
- `lastIndexOf(Object o)`: Returns the index of the last occurrence of the specified element.
- `iterator()`: Returns an iterator for the list elements.
- `descendingIterator()`: Returns an iterator that iterates in reverse order.

Reverse a LinkedList

Problem: Given the head of a singly linked list, reverse the list in-place. This means you should modify the linked list itself without using extra space proportional to the length of the list.

Explanation: We can reverse a linked list iteratively by keeping track of the previous, current, and next nodes.

Initialize prev to None, current to head.

While current is not None:

Store the next node of current.

Set the next pointer of current to prev.

Move prev to current.

Move current to the stored next node.

After the loop, prev will be the new head of the reversed list.

Input Sample:

LinkedList: 1 -> 2 -> 3 -> 4 -> 5 -> None

Output Sample:

LinkedList: 5 -> 4 -> 3 -> 2 -> 1 -> None

Rotate a LinkedList

Problem: Given the head of a linked list and an integer k , rotate the list to the right by k places.

Explanation:

Find the length of the linked list. Iterate through the list and count the number of nodes. Let the length be len .

Handle edge cases. If the list is empty or k is a multiple of len , the list remains unchanged. Return the original head.

Calculate the effective rotation. The effective number of rotations needed is $k \% len$.

Find the new head and the new tail.

Traverse the list to find the $(len - k \% len)$ -th node from the beginning. Let this node be the new tail.

The node after the new tail will be the new head.

Perform the rotation:

Set the next pointer of the new tail to None to break the original list.

Traverse to the end of the original list (which is now before the original head in the rotated version).

Set the next pointer of the original tail to the original head.

Return the new head found in step 4.

Input Sample:

LinkedList: 1 -> 2 -> 3 -> 4 -> 5 -> None, $k = 2$

Output Sample:

LinkedList: 4 -> 5 -> 1 -> 2 -> 3 -> None

Explanation: Rotating to the right by 2 places means the last two elements (4 and 5) move to the front.

Input Sample:

LinkedList: 1 -> 2 -> 3 -> None, $k = 4$

Output Sample:

LinkedList: 3 -> 1 -> 2 -> None

Explanation: Length is 3. Effective rotation is $4 \% 3 = 1$. The last element (3) moves to the front.

Tree in Java - Implementation and Methods

What is a Tree?

- A Tree is a non-linear data structure consisting of nodes connected by edges.
- Each node contains data, and nodes are connected in a hierarchical manner.
- Common tree types: Binary Tree, Binary Search Tree, AVL Tree, Red-Black Tree.

Java Collection Framework Tree Implementations:

- Java provides TreeSet and TreeMap as part of the java.util package.
- TreeSet implements a self-balancing binary search tree (Red-Black Tree).
- TreeMap is a map that maintains keys in sorted order using a Red-Black Tree.

Example: Implementing TreeSet

```
import java.util.TreeSet;

public class TreeSetExample {

    public static void main(String[] args) {

        TreeSet<Integer> treeSet = new TreeSet<>();

        treeSet.add(10);

        treeSet.add(20);

        treeSet.add(5);

        System.out.println("TreeSet: " + treeSet);

        // Output: [5, 10, 20]

        System.out.println("First Element: " + treeSet.first());

        System.out.println("Last Element: " + treeSet.last());
```

```
    }  
}
```

Common TreeSet Methods:

- `add(E e)`: Adds the specified element to the set.
- `remove(Object o)`: Removes the specified element.
- `contains(Object o)`: Checks if the set contains the specified element.
- `first()`: Returns the first (lowest) element.
- `last()`: Returns the last (highest) element.
- `pollFirst()`: Retrieves and removes the first element.
- `pollLast()`: Retrieves and removes the last element.
- `higher(E e)`: Returns the smallest element greater than e.
- `lower(E e)`: Returns the largest element less than e.
- `ceiling(E e)`: Returns the least element $\geq e$.
- `floor(E e)`: Returns the greatest element $\leq e$.

Example: Implementing TreeMap

```
import java.util.TreeMap;  
  
public class TreeMapExample {  
    public static void main(String[] args) {  
        TreeMap<Integer, String> treeMap = new TreeMap<>();  
        treeMap.put(3, "Apple");  
        treeMap.put(1, "Banana");  
        treeMap.put(2, "Cherry");  
  
        System.out.println("TreeMap: " + treeMap); // Output:  
        {1=Banana, 2=Cherry, 3=Apple}  
  
        System.out.println("First Key: " + treeMap.firstKey());  
        System.out.println("Last Key: " + treeMap.lastKey());  
    }  
}
```

```
    }  
}
```

Common TreeMap Methods:

- `put(K key, V value)`: Inserts a key-value pair.
- `remove(Object key)`: Removes the mapping for a key.
- `get(Object key)`: Returns the value associated with the key.
- `firstKey()`: Returns the first (lowest) key.
- `lastKey()`: Returns the last (highest) key.
- `higherKey(K key)`: Returns the least key strictly greater than the given key.
- `lowerKey(K key)`: Returns the greatest key strictly less than the given key.
- `ceilingKey(K key)`: Returns the least key greater than or equal to the given key.
- `floorKey(K key)`: Returns the greatest key less than or equal to the given key

Validate Binary Search Tree

Problem: Given the root of a binary tree, determine if it is a valid Binary Search Tree (BST). A valid BST is defined as follows:¹

- The left subtree of a node contains only nodes with keys less than the node's key.
- The right subtree of a node contains only nodes with keys greater than the node's key.
- Both the left and right subtrees must also be binary search trees.²

Explanation: We can validate a BST using an inorder traversal. In an inorder traversal of a valid BST, the nodes are visited in ascending order.

1. Perform an inorder traversal of the binary tree, storing the visited node values in a list.
2. Iterate through the list of visited values and check if they are in strictly increasing order. If they are, the tree is a valid BST; otherwise, it is not.

Alternatively, we can use recursion with a range. For each node, we check if its value is within the valid range defined by its ancestors.

Recursive Approach with Range:

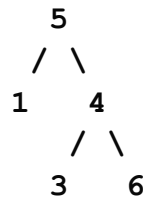
1. Create a helper function `isValidBSTHelper(node, minVal, maxVal)`.
2. Base case: If node is None, return True.
3. Check if `node.val` is within the range (`minVal`, `maxVal`). If not, return False.
4. Recursively call `isValidBSTHelper` for the left child with the updated `maxVal` as `node.val`, and for the right child with the updated `minVal` as `node.val`.
5. Return True if both recursive calls return True.
6. Call the helper function with the root and initial `minVal` as negative infinity and `maxVal` as positive infinity.

Input Sample (Valid BST):

```
    2
   / \
  1   3
```

Output Sample:True

Input Sample (Invalid BST):



Output Sample:False

```
package Tree_Examples;
```

```
import java.util.Scanner;
```

```
class Node {
```

```
    int data;
```

```
    Node left, right;
```

```
    Node(int data) {
```

```
        this.data = data;
```

```
        left = right = null;
```

```
    }
```

```
}
```

```
public class Tree_Class {
```

```
    Node root;
```

```
    void addManual(Node r, int key, int data, String
direction) {
```

```
        if (r == null) {
```

```

        System.out.println("Node with key " + key + " not
found.");

        return;
    }

    if (r.data == key) {
        Node newNode = new Node(data);

        if (direction.equalsIgnoreCase("L")) {
            if (r.left == null) {
                r.left = newNode;

                System.out.println("Added " + data + " to
the LEFT of " + key);
            } else {
                System.out.println("Left child already
exists.");
            }
        } else if (direction.equalsIgnoreCase("R")) {
            if (r.right == null) {
                r.right = newNode;

                System.out.println("Added " + data + " to
the RIGHT of " + key);
            } else {
                System.out.println("Right child already
exists.");
            }
        } else {
            System.out.println("Invalid direction. Use 'L'
or 'R'.");
        }
    }

```

```

    } else {

        addManual(r.left, key, data, direction);

        addManual(r.right, key, data, direction);

    }

}

void inorder(Node r) {

    if (r != null) {

        inorder(r.left);

        System.out.print(r.data + " ");

        inorder(r.right);

    }

}

public static void main(String[] args) {

    Tree_Class obj = new Tree_Class();

    Scanner in = new Scanner(System.in);

    System.out.println("Enter root node data:");

    int rootData = in.nextInt();

    obj.root = new Node(rootData);

    while (true) {

        System.out.println("Enter parent node, new node
data, and direction (L/R) or -1 to stop:");

        int key = in.nextInt();

        if (key == -1) break;

```

```
        int data = in.nextInt();

        String direction = in.next();

        obj.addManual(obj.root, key, data, direction);

        System.out.println("Current Tree (Inorder
Traversal):");

        obj.inorder(obj.root);

        System.out.println();

    }

}

}
```